None

Sei un ingegnere del software quantistico esperto di NVIDIA CUDA-Q, logistica industriale OT (sistemi Elettric80 / SM.I.LE80) e conformità NIS2. Il mio software quantistico "JOKER 80 QUANTUM" gira su un cluster NVIDIA Grace Hopper GH200 (simulatori cuStateVec/cuTensorNet) ma fallisce la compilazione, contiene bug di logica quantistica e i report di log JSON sono totalmente disallineati rispetto al codice Python.

Riscrivi il codice Python/CUDA-Q di ciascun calcolo correggendo TUTTI gli errori sintattici, quantistici e di data-mapping elencati di seguito. Assicurati che l'Host Code Python sputi fuori un JSON con chiavi e dati perfettamente coerenti con la telemetria PLC del rispettivo report.

Ecco la mappatura dettagliata degli errori presenti in ogni singolo calcolo da correggere:

CALCOLO 1: Quantum Boltzmann Machine (QBM) per Inbound Congestion
1. Errore di Sintassi CUDA-Q: I gate h(), ry(), x.ctrl(), rz(), mz() sono chiamati nel namespace globale. Vanno corretti obbligatoriamente in cudaq.h(), cudaq.ry(), cudaq.x.ctrl(), cudaq.rz(), cudaq.mz().
2. Errore Parametro Sample: L'argomento 'shots_count=1000' in cudaq.sample() non esiste. Va rinominato in 'shots=1000'.
3. Disallineamento Dati PLC nel Report: Nella Sezione 1 l'input dichiara camion_attesa=12, minuti_ritardo=45, saturazione_wms=88.5. Nella Sezione 2 del report (JSON), i dati elaborati crollano arbitrariamente a 6 camion, 48 min, 89.4% saturazione. Correggere il dizionario di ritorno dell'Host Code per mappare i dati reali.
4. Errore Matematico di Congestione: Con i dati reali (12, 45, 88.5), la formula genera 96.0% di congestione, mentre il report dichiara falsamente 77.4% a causa della perdita di dati di cui sopra.

CALCOLO 2: Quantum Monte Carlo Risk Analysis per Fermo Linee
1. Errore di Sintassi CUDA-Q: Mancano i prefissi del modulo sui gate quantistici nel kernel. Correggere in cudaq.ry(), cudaq.x.ctrl(), cudaq.h(), cudaq.z(), cudaq.mz().
2. Errore Parametro Sample: Sostituire 'shots_count=1000' con 'shots=1000' in cudaq.sample().
3. Discrepanza Dati nel Report: Il report calcola un Costo Stimato (VaR 95%) fisso di €5850 nel testo, ma la formula nel codice Python genera un

output matematico dinamico differente basato sui minuti di ritardo. Allineare il dizionario di output.

CALCOLO 3: Quantum Integer Programming (QUBO) - Schedulazione Baie 24h
1. Assenza di Codice Quantistico: Il codice è un algoritmo puramente classico basato su matrici NumPy (np.zeros, np.trace). Non alloca qubit, non usa kernel e non invoca CUDA-Q.
2. Incoerenza Tecnologica nel Report: L'architettura QPU dichiarata nel report menziona "D-Wave Annealer / QPU Gate-Based QAOA", ma il codice non implementa né l'embedding per Annealer né il circuito variazionale QAOA. Riscrivere il calcolo integrando un vero algoritmo variazionale quantistico in CUDA-Q per risolvere la matrice cost/QUBO Q.

CALCOLO 4: Quantum Walk-based Clustering per Controllo Qualità Lotti
1. Errore di Sintassi CUDA-Q: I gate h(), ry(), x.ctrl(), mz() all'interno del kernel non hanno il prefisso 'cudaq.'.
2. Errore Parametro Sample: Cambiare 'shots_count=1000' in 'shots=1000'.
3. Allocazione Qubit Errata: Il coin quantistico viene allocato come singolo qubit 'coin = cudaq.qubit()', ma nel loop viene applicato l'operatore controllato 'x.ctrl(coin, q[0])' in modo iterativo senza ripreparare correttamente lo stato della moneta o senza usare un registro coin multi-qubit coerente con i passi (steps=3) del Quantum Walk.
4. Alterazione Dati Telemetria: L'input PLC inserito indica umidita_rilevata=13.2 e spessore_micro=45.2. Nel JSON del report i dati elaborati scendono a umidità=12 e spessore=42, troncando i decimali e falsando l'indice di conformità.

CALCOLO 5: Quantum Kernel Ridge Regression per Magazzino SmartStore
1. Errore di Sintassi CUDA-Q: Mancano i prefissi del modulo sui gate h(), rz(), x.ctrl(), mz() nel kernel.
2. Errore Parametro Sample: Modificare 'shots_count=1000' in 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice Python esegue una regressione quantistica di kernel per stimare l'usura delle guide del magazzino automatico (input: temperatura e saturazione). Il report allegato descrive invece un algoritmo di "Post-Quantum Cryptographic

Hashing" con output SHA-3/256 per sigillare lotti di bobine di plastica. I due moduli sono completamente scambiati. Riscrivere l'output del codice o il report per garantire totale coerenza.

CALCOLO 6: QAOA per Schedulazione Missioni Traslo-Elevatori SmartStore
1. Errore di Sintassi CUDA-Q: Tutti i gate del circuito QAOA (h, x.ctrl, rz, rx, mz) mancano del prefisso 'cudaq.'.
2. Errore Parametro Sample: Modificare 'shots_count=1000' in 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice ottimizza le missioni in coda dei trasloelevatori riducendo i tempi di ciclo e calcolando il risparmio energetico in kWh. Il report allegato descrive invece il "Quantum Digital Twin Alignment & Bin Packing" per l'assegnazione di celle 3D nel magazzino in base alle classi di rotazione delle merci. I dati del JSON non corrispondono alle variabili calcolate (es. risparmio_kwh non compare nel report).

CALCOLO 7: Algoritmo di Grover per Rilocazione Pallet SmartStore
1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' su tutti i gate (h, x, x.ctrl, mz) dell'oracolo e del diffusore di Grover.
2. Errore Parametro Sample: Sostituire 'shots_count=1000' con 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice esegue una ricerca di Grover per identificare le coordinate (Corridoio/Livello) della cella ottimale di stoccaggio. Il report allegato descrive invece una "Quantum Hopfield Network" per l'analisi del fattore di sicurezza strutturale e dello stress meccanico dei pattini (in bar e MPa). C'è una totale inversione logica delle funzioni.

CALCOLO 8: Quantum Topological Data Analysis (TDA) per Scorte Magazzino
1. Totale Assenza di Codice Quantistico: Il file si dichiara un algoritmo Quantum TDA (Lloyd-Silberstein-Blunt), ma è composto solo da due righe classiche di divisione intera in Python nativo (// 240 e // 900). Non invoca CUDA-Q, non ha qubit, non ha feature map quantistica.

2. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice analizza l'omogeneità topologica e i numeri di Betti dell'inventario. Il report descrive invece una rotta quantistica ottimizzata per il problema del commesso viaggiatore (TSP / Quantum Walk Route Exploration) con tanto di sequenza di prelievo a step (Step 1: PLT_C, ecc.) e metri lineari risparmiati, dati totalmente assenti nel codice classico sorgente.

CALCOLO 9: Quantum Circuit Learning (QCL) per Dispatching Navette
1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' su tutti i gate (ry, rx, rz, x.ctrl, mz) del circuito variazionale.
2. Errore Parametro Sample: Sostituire 'shots_count=1000' con 'shots=1000'.
3. Rischio Out of Index e compilazione JIT: Il ciclo per i layer parametrici rx(theta_pesi[i]) e rz(theta_pesi[i+4]) assume una dimensione fissa della lista di float passata dall'host. In CUDA-Q, per evitare crash di compilazione JIT, le liste di parametri devono essere mappate esplicitamente tramite tipi statici o cudaq.Array.
4. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice assegna il numero di navette AGV/LGV necessarie per gestire i fardelli al minuto. Il report allegato descrive invece una "Quantum Graph Neural Network (QGNN)" per il raggruppamento ordini (Picking & Batching) nel modulo WMS, misurando l'efficienza dei lotti in percentuale, ignorando il dispatching delle navette.

CALCOLO 10: Multi-Qubit Variational Routing per Flotta LGV/AGV
1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' su h(), x.ctrl(), rz(), mz() nel kernel del deadlock-prevention.
2. Errore Parametro Sample: Sostituire 'shots_count=1000' con 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice Python calcola il rischio di blocco e le corsie di svincolo (Bypass_Nord, Corsia_B3) per evitare collisioni di navette AGV. Il report allegato descrive invece il modulo "3D Volumetric Loader" per calcolare la saturazione volumetrica del cassone di un camion in percentuale e il bilanciamento statico del peso degli assi anteriore/posteriore in kg. Il mapping è completamente invertito.

```
### CALCOLO 11: Quantum Phase Estimation (QPE) per Batteria LGV
1. Errore Sintattico Bloccante Gate Controllato: Nel kernel viene usato
il comando 'crz(fase_degrado, q_counting[i], q_eigen)'. Questo comando
genera un errore di compilazione fatale. In CUDA-Q i gate controllati si
applicano tramite la proprietà del gate base. La sintassi corretta è:
cudaq.rz.ctrl(fase_degrado, q_counting[i], q_eigen).
2. Errore di Sintassi Generale: Mancano i prefissi 'cudaq.' su tutti gli
altri gate (x, h, mz).
3. Errore Parametro Sample: Sostituire 'shots_count=1000' con
'shots=1000'.
4. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice Python esegue
l'algoritmo QPE per stimare lo stato di salute (SoH) e l'autonomia
residua della batteria di una navetta LGV. Il report descrive invece la
firma crittografica post-quantum di contratti di trasporto e-CMR per
spedizioni in Germania tramite algoritmi ibridi Dilithium/Falcon,
applicando un salt esadecimale. I moduli d'uso sono totalmente
scollegati.

---

### CALCOLO 12: CVQC per Fasciatore Bema Silkworm Elettric80
1. Assenza Totale di Codice Quantistico: Il codice dichiara di usare un
modello continuo di squeezing quantistico (CVQC) per la tensione del film
plastico, ma esegue solo normali operazioni matematiche classiche in
Python (min, np.exp). Non inizializza alcun simulatore quantistico a
variabili continue (come il backend Gaussian di Strawberry
Fields/CUDA-Q), rendendolo un codice interamente classico.
```

Usa il codice con cautela.

2. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice calcola i grammi di film risparmiati per pallet e il rischio di rottura del film stretch del fasciatore Bema. Il report allegato descrive invece l'applicazione di algoritmi di "Quantum Game Theory" per calcolare l'Equilibrio di Nash nell'assegnazione delle baie di carico (Dock & Slot Scheduler) nel piazzale (YMS), misurando i costi orari di attesa dei camion.

CALCOLO 13: Quantum State Tomography (QST) per Tensione Angoli Pallet

1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' sui gate h(), rz(), mz() nel kernel della tomografia.
2. Errore Parametro Sample: Cambiare 'shots_count=1000' in 'shots=1000'.

3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice analizza l'omogeneità della tensione del film agli angoli del pallet tramite ricostruzione dello stato quantistico. Il report allegato descrive invece un algoritmo "Quantum K-Means" per la gestione del traffico del piazzale e la saturazione dei buffer polmone del magazzino automatico (Modulo YMS).
4. Corruzione dei dati PLC: Nella Sezione 1 l'input dichiara 14 camion in attesa e saturazione buffer a 0.75. Nel JSON dei dati elaborati, il sistema registra invece 6 camion e saturazione a 0.88. Allineare i flussi di input/output.

CALCOLO 14: Quantum Gaussian Mixture Model per Vibrazioni Cuscinetti

1. Assenza Totale di Codice Quantistico: Il codice si dichiara un modello quantistico di miscela gaussiana nello spazio di Hilbert per la manutenzione predittiva dei cuscinetti, ma è un semplice script classico Python con calcoli aritmetici elementari. Nessun qubit allocato, nessun target quantistico nvidia chiamato.
2. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice analizza le vibrazioni del mandrino e i gradi RMS per stimare le ore residue prima di un guasto meccanico. Il report allegato descrive invece l'esecuzione di una vera "Quantum Fourier Transform (QFT)" sul modulo Bema Wrapping Controller per analizzare lo spettro delle frequenze armoniche in Hz delle tavole rotanti del fasciatore.

CALCOLO 15: Variational Quantum Linear Solver (VQLS) per Stabilità Pallet

1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' sui gate ry(), x.ctrl(), mz() all'interno del kernel VQLS.
2. Errore Parametro Sample: Sostituire 'shots_count=1000' con 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice risolve un sistema lineare (VQLS) per determinare lo sbilanciamento del baricentro degli strati del pallet e prevenire il ribaltamento del carico. Il report descrive invece un modello di classificazione "Quantum Support Vector Machine (QSVM)" per stimare lo sforzo unitario in MPa e il rischio di strappo del film polietilene stretch nel modulo ECS. I dati di output non coincidono.
4. Alterazione Dati Telemetria PLC: L'input della telemetria indica tensione=148.5 N e velocità=12.4 m/s. I dati elaborati nel JSON vengono forzati a numeri tondi arbitrari (150 N e 11.2 m/s), violando l'integrità del dato OT richiesta dalla NIS2.

CALCOLO 16: Quantum Fourier Transform (QFT) per Risonanza Rulliere

1. Errore Sintattico Bloccante Gate Controllato: All'interno della routine della QFT a 4 qubit viene chiamato il gate 'crz(np.pi / ..., q[j], q[i])'. In CUDA-Q questo comando fallisce. Sostituire obbligatoriamente con la sintassi corretta: cudaq.rz.ctrl(np.pi / (2 ** (j - i)), q[j], q[i]).
2. Errore di Sintassi Generale: Mancano i prefissi 'cudaq.' su h(), rz(), mz() nel kernel.
3. Errore Parametro Sample: Cambiare 'shots_count=1000' in 'shots=1000'.
4. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice applica la QFT per rilevare armoniche di risonanza pericolose sulle rulliere comandate da inverter. Il report allegato descrive invece un algoritmo "Dynamic Graph QAOA & QRL" per calcolare l'indice di fluidità della rete di navigazione laser dei veicoli AGV/LGV (SDM Routing & Traffic Engine), analizzando nodi stradali bloccati e coordinate di posizionamento delle navette.

CALCOLO 17: Quantum Reinforcement Learning (QRL) per Sincronizzazione Linee

1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' su tutti i gate (ry, x.ctrl, rz, mz) del policy kernel.
2. Errore Parametro Sample: Cambiare 'shots_count=1000' in 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice Python implementa un algoritmo QRL per determinare le variazioni di velocità (delta BPM) per sincronizzare il flusso dei fardelli sulle linee di imbottigliamento/confezionamento. Il report descrive invece un motore di allocazione task basato su "Quantum Bipartite Matching" per accoppiare le missioni urgenti di fabbrica ai pacchi batteria delle navette LGV in base al loro stato di carica (SoC), temperatura e Wh rigenerati.

CALCOLO 18: Quantum Graph Neural Network (QGNN) per Cross-Docking

1. Assenza di Codice Quantistico: Lo script si dichiara una Rete Neurale Quantistica a Grafi (QGNN) per ottimizzare i flussi di evacuazione diretta delle merci (Cross-Docking), ma non contiene alcuna istruzione CUDA-Q. Calcola la percentuale di evacuazione tramite formule puramente matematiche classiche.
2. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice calcola la capacità di evacuazione delle baie di carico outbound e i pallet risparmiati dallo stoccaggio a terra. Il report allegato descrive invece un algoritmo "Variational Quantum Eigensolver (VQE)" applicato alla cinematica dei giunti dei robot di pallettizzazione (Robotic Palletizer Dynamics), misurando la tenuta del vuoto delle pinze di presa in bar, la coppia dei motori in Nm e la corrente dei joint in Ampere.

CALCOLO 19: Quantum Hopfield Network per Assegnazione Camion Outbound

1. Errore di Sintassi CUDA-Q: Mancano i prefissi 'cudaq.' sui gate h(), x.ctrl(), mz() presenti nell'associative kernel.
2. Errore Parametro Sample: Cambiare 'shots_count=1000' in 'shots=1000'.
3. DISALLINEAMENTO TOTALE REPORT-CODICE: Il codice mappa l'assegnazione dei camion in uscita (Outbound) per ottimizzare la saturazione volumetrica del cassone in base agli ordini in coda nel WMS. Il report descrive invece un modulo del sistema ECS denominato "Fast-Charge & Supercap Power Balancer" che usa un algoritmo "Quantum Knapsack" per fare peak-shaving della rete elettrica delle stazioni di ricarica rapida, monitorando la temperatura delle piastre e la potenza erogata in kW.

Genera per ciascun calcolo il codice Python corretto, pulito, aderente alle specifiche CUDA-Q più recenti e assicurati che i dizionari restituiti dalle funzioni rispecchino fedelmente i requisiti applicativi e i parametri di telemetria industriale (PLC) corretti.